Fine Taxonomy

Fine Taxonomy and Crop Classifier

Carrier, vehicle make and package size for ARGUS: datasets, simulation results, the model in training, how to run everything, and the path to a production node.

Branch `feat/fine-taxonomy` (commits `a9234ed`, `aa6faae`). Status as of 2026-10-03.

1. Summary

- **Goal.** Go past "vehicle" and "package" to "UPS truck", "the resident's Nissan", "USPS regular mail", "large parcel", so the federated head can tell a routine delivery from a real anomaly.
- **How.** A second-stage crop classifier looks inside each YOLO vehicle or package box. Its answers become 16 extra features for the federated head (19 coarse + 16 = 35).
- **Simulation result.** With a 90% accurate crop classifier, FedAvg balanced accuracy rises from 0.860 to 0.913, and accuracy on the events only the subtype can decide rises from 0.40 to 0.61. Below about 75% classifier accuracy the benefit mostly disappears.
- **Frame rate.** A 6 ms crop stage (an estimate, not a measurement) drops the simulated node from 14.6 to 13.3 FPS. The 12 FPS target still holds through a training round.
- **Sponsor finding.** Local-only training beat FedAvg in every run. One shared model cannot learn that each site's household drives different cars. Per-site fine-tuning after FedAvg is needed.
- **Real model.** The make classifier is trained (Stanford Cars, CPU, 12 epochs, about 55 minutes). Test accuracy is 0.832 over 49 makes and 0.917 after collapsing to the 7 ARGUS slots; on the six tracked makes alone, recall is 0.71 to 0.85 (about 0.76 weighted). Plugged back into the sim at 0.76, FedAvg balanced accuracy still rises from 0.860 to 0.894.
- **Speed.** One crop takes 15.8 ms on the x86 dev CPU. A Pi CPU is several times slower, so the crop stage has to run on the Hailo.

2. Datasets

Loaded

Only one dataset is downloaded and in use.

Dataset	What it holds	Location	Size on disk	License
Stanford Cars (Hugging Face parquet mirror)	16,185 photos, 196 make-model-year classes, collapsed to 49 makes	`D:\argus-data\stanford-cars`	1.9 GB	Not stated on the mirror. The original Stanford release is for research use; treat as non-commercial.

Derived from it:

Folder	Contents
`D:\argus-data\crops\make`	7,330 train / 813 val / 8,040 test images, resized to a 256 px short side, one folder per make (467 MB)
`D:\argus-data\models\crop_make`	`model.pt` (trained make classifier, 17 MB), `model.onnx` (for the Hailo compiler), `metrics.json` (test accuracy, per-epoch history, 49 x 49 confusion matrix)
`D:\argus-data\train_make.log`	Training log

The 49 makes include all six ARGUS tracks (Nissan, Toyota, Honda, Ford, Chevrolet, Tesla); the other 43 collapse to "other".

The simulator's own dataset is synthetic: `sim/workload_fine.py` generates per-site events in memory, so nothing is downloaded for it.

Identified, not yet downloaded

Task	Dataset	Labels	License	Blocker
Carrier	USPS Merge	Amazon, FedEx, Other-Vehicles, UPS, USPS-Truck	not verified	Roboflow login
Carrier (extra)	Delivery Trucks	delivery truck classes	not verified	Roboflow login
Package size	mailers and boxes 2	mailer, box	not verified	Roboflow login
Package size	Envelope	envelope	not verified	Roboflow login
Package size	Parcels and Boxes	parcel, box	not verified	Roboflow login
Doorstep test images	Roboflow Packages	one class, 26 raw images	Public domain	Too small to train on; no size labels
Make (larger)	VMMRdb	291,752 images, 9,170 classes	MIT, cite Tafazzoli et al. 2017	Not needed yet; the commercial-safe replacement for Stanford Cars

Roboflow blocks automated page reads, so the carrier and package-size class lists above come from search results, and their image counts and licenses are unverified.

Gaps no public dataset fills

- **Large vs small parcel.** Public sets label "box", not size.
- **Carrier on a package**, such as a UPS label on a box at the door.
- **Camera angle.** Stanford Cars is mostly eye-level side and 3/4 views. ARGUS cameras look down. Accuracy on Stanford's test set will overstate accuracy on real footage until ARGUS's own clips are labelled.

3. What was built

File	Purpose
`sim/workload_fine.py`	35-feature synthetic events: carrier, make, package size, crop confidence; 4 sites incl. new `curbside`
`sim/compare_taxonomy.py`	Coarse vs fine, FedAvg vs local-only, swept over classifier accuracy
`sim/workload.py`	Split into sample / label / assemble; output pinned byte-for-byte
`sim/head.py`	Input size set per taxonomy (19 or 35)
`sim/fl.py`, `sim/run_node.py`	`--taxonomy fine`, `--classifier-acc`, `curbside` site
`sim/profiles.py`	`pi5-hailo-fine` profile with a 6 ms crop stage (estimate)
`training/crop/prepare_stanford_cars.py`	Stanford Cars parquet to make folders
`training/crop/prepare_roboflow.py`	Roboflow COCO export to carrier or size folders
`training/crop/train.py`	MobileNetV3-Large fine-tuning, metrics, ONNX export
`training/crop/taxonomy.py`	Dataset class names to ARGUS slots
`src/detection/crop_classifier.py`	Runtime classifier, PyTorch and ONNX backends
`src/detection/__init__.py`	`Detection.subtype`, `attach_crop_classifier()`, called after YOLO
`tests/test_sim_fine_taxonomy.py`, `tests/test_crop_training.py`, `tests/test_crop_classifier.py`	37 new tests

The runtime hook is inert until a model is attached, and any failure leaves `subtype` empty rather than dropping the detection. The 138 existing detection-suite tests still pass, and the coarse generator that `src/federated/features.py` mirrors is unchanged.

4. Simulation design

Features (35). The 19 coarse features, then:

Slots	Values	Applies to
Carrier	none, ups, usps, fedex, amazon	vehicles, packages
Make	nissan, toyota, honda, ford, chevrolet, tesla, other	vehicles
Package size	mail, small, large (mail is USPS only)	packages
Crop confidence	0 to 1	vehicles, packages

Sites.

Site	Household cars	Character
driveway	Nissan, Toyota	Residents and deliveries all day
backdoor	none	Deliveries only; any vehicle is an anomaly
yard	none	Mostly animals
curbside (new)	Honda, Ford	Mailbox, USPS daily, residents park

Rules, decided from the true subtype.

- A household car is routine at any hour and never counts as loitering.
- A carrier vehicle or package is routine only inside its delivery window: UPS 9-21, USPS 9-18, FedEx 8-20, Amazon 8-22.
- Everything else follows the original coarse rule.

The simulated crop classifier reports the true subtype with probability `--classifier-acc`, otherwise a wrong one. The head sees the prediction; the label follows the truth.

Delivery windows, per-site subtype mixes and geometry multipliers are simulation assumptions, not sourced figures.

5. Simulation results

Coarse vs fine, by classifier accuracy

6,000 events per site, 6 rounds x 2 epochs, seed 1. "Subtype-decided" = events where the true label differs from what the coarse rule alone would say.

Classifier acc	Training	Balanced acc, coarse	Balanced acc, fine	Subtype-decided, coarse	Subtype-decided, fine
1.00	FedAvg	0.860	0.908	0.400	0.636
1.00	Local	0.921	0.962	0.407	0.708
0.90	FedAvg	0.860	0.913	0.400	0.613
0.90	Local	0.921	0.949	0.407	0.646
0.75	FedAvg	0.860	0.893	0.400	0.557
0.75	Local	0.921	0.937	0.407	0.561
0.50	FedAvg	0.860	0.871	0.400	0.420
0.50	Local	0.921	0.919	0.407	0.469

Per-slice accuracy at 90% classifier accuracy

Slice	n	FedAvg coarse	FedAvg fine	Local coarse	Local fine
Household car, night	19	0.000	0.105	0.579	1.000
Stranger car, night	25	1.000	0.960	0.560	0.760
Carrier truck, in window	305	0.705	0.813	0.951	0.941
Carrier truck, out of window	118	0.881	0.983	0.373	0.873
USPS regular mail	121	0.785	0.934	0.959	0.942
Large parcel, carrier	346	0.925	0.968	0.928	0.951

The two night-time car slices have 19 and 25 events; treat those rows as rough.

Why FedAvg fails household cars. The driveway's household drives a Nissan and a Toyota, the curbside's a Honda and a Ford. A single global model with no site identity cannot hold both rules, so it scores 0% where local training scores 100%. Personalisation (local fine-tuning after FedAvg) is the standard fix.

Full Flower run, 4 clients over gRPC

5 rounds, 8,000 events per site, 4 epochs per round, head 211,589 parameters (budget about 200K).

Round	Accuracy	Balanced acc
1	0.908	0.849
5	0.914	0.864

Final per-site balanced accuracy: yard 0.912, curbside 0.913, driveway 0.905, backdoor 0.727.

Frame rate, 1 client, 4 cores

Profile	Budget ceiling	Held while training	Target 12 FPS
pi5-hailo	14.9 FPS	14.62 FPS	held
pi5-hailo-fine (+6 ms crop)	13.7 FPS	13.33 FPS	held

Stage times are Rev 2.2 Table 11 budgets, not Pi measurements; the crop line is an estimate.

6. Make classifier training

MobileNetV3-Large, ImageNet weights, 224 px, 49 makes, class-balanced sampling, label smoothing 0.1, 12 epochs, CPU only (12 threads, about 4-5 minutes an epoch).

Epoch	Train loss	Val accuracy
1	3.744	0.137
2	2.293	0.309
3	1.448	0.530
4	1.178	0.579
5	1.009	0.684
6	0.940	0.720
7	0.896	0.770
8	0.857	0.786
9	0.827	0.813
10	0.812	0.824
11	0.808	0.824
12	0.805	0.827

Test set (8,040 images).

Measure	Value
Accuracy, 49 makes	0.832
Accuracy, 7 ARGUS slots	0.917
Weighted recall, six tracked makes	about 0.76

ARGUS slot	Test images	Recall
other (43 makes)	6,100	0.966
honda	160	0.850
tesla	38	0.842
ford	514	0.825
nissan	170	0.771
toyota	164	0.744
chevrolet	894	0.709

The 0.917 is flattered by "other", which is 76% of the test set and easy to get right. The tracked makes, the ones the rules depend on, sit at about 0.76. These are Stanford Cars photos at eye level, so expect lower on a downward-looking ARGUS camera until real footage is added.

Fed back into the sim (`compare_taxonomy.py --acc 0.92,0.76`):

Classifier acc	Training	Balanced acc, coarse to fine	Subtype-decided, coarse to fine
0.92	FedAvg	0.860 to 0.909	0.400 to 0.616
0.92	Local	0.921 to 0.952	0.407 to 0.672
0.76	FedAvg	0.860 to 0.894	0.400 to 0.518
0.76	Local	0.921 to 0.933	0.407 to 0.548

Runtime check. `CropClassifier.from_dir` loads the model and labels real test crops; one crop takes 15.8 ms on the x86 dev CPU (4 threads). On a Pi CPU that would be several times slower, well past the 6 ms the `pi5-hailo-fine` profile budgets for the Hailo.

7. How to run the sims

From `C:\Users\colab\prometheus_workspace\argus-fl` on branch `feat/fine-taxonomy`.

Accuracy: does the subtype help? About 90 seconds.

```
python sim/compare_taxonomy.py
python sim/compare_taxonomy.py --acc 0.85
python sim/compare_taxonomy.py --acc 1.0,0.9 --rounds 4 --samples 4000
```

Writes `sim/_run/taxonomy_report.md` and `taxonomy_report.json`.

Frame rate: does the node keep up while training?

```
python sim/run_node.py --taxonomy fine --profile pi5-hailo-fine --clients 1 --cores 4
python sim/run_node.py --taxonomy fine --profile pi5-hailo --clients 1 --cores 4
```

Use `--clients 1` for frame rate. More clients all train on the same PC.

Learning across sites: FedAvg over all four.

```
python sim/run_node.py --taxonomy fine --clients 4 --rounds 5 --samples 8000 --epochs 4
```

Flags. `--classifier-acc 0.8`, `--site curbside`, `--profile pi5-threat-on-cpu`, `--port 0` (no metrics endpoint). Without `--taxonomy fine` everything runs exactly as before.

Tests.

```
python -m pytest tests/test_sim_fine_taxonomy.py tests/test_crop_training.py
tests/test_crop_classifier.py -q
```

Training the classifiers.

```
python training/crop/prepare_stanford_cars.py --out D:/argus-data/crops/make
python training/crop/train.py --task make --data D:/argus-data/crops/make --out D:/argus-
data/models/crop_make
python training/crop/train.py --out D:/argus-data/models/crop_make --export-only   # re-export ONNX
only
python training/crop/prepare_roboflow.py --task carrier --src D:/argus-data/roboflow/usps-merge --
out D:/argus-data/crops/carrier
python training/crop/train.py --task carrier --data D:/argus-data/crops/carrier --out D:/argus-
data/models/crop_carrier
```

8. Path to production

Step	Work	Pass bar	Owner / blocker
1. Data	Download Roboflow carrier and package sets; label a few hundred crops from ARGUS's own clips (carrier trucks, mail vs small vs large, resident cars)	At least 75% accuracy on ARGUS footage	Needs a Roboflow login and labelling time
2. Licensing	Retrain make on VMMRdb (MIT) plus own data before selling nodes	No research-only data in a shipped model	Before any sale at $299-499
3. Hailo	`model.onnx` to `.hef` with the Hailo Dataflow Compiler; add a HailoRT backend; measure per-crop time on the Pi	Measured crop time keeps 8-12 FPS	x86 Linux or WSL2, Hailo developer account; confirm Hailo-8 vs Hailo-10H
4. Shadow mode	Config setting such as `CROP_MODELS_DIR`; attach in `src/api/app.py`; store subtype on events; show it on the dashboard; head does not use it yet	1-2 weeks of logged subtypes checked against reality	Code work, not started
5. Federated head	`features.py` and `collector.py` to 35 features; migrate the training store; update the aggregator; add per-site fine-tuning	Fine head beats coarse on held-out real events	Team decision; all nodes upgrade together
6. Existing blockers	Auth on trial and dashboard routes; wire `robust.py`; loss tie-break in the validation gate; recompile the `.hef` after each round	All closed	Already open before this work

Against the calendar (code freeze 2026-11-08, showcase 2026-11-22): steps 1, 3 and 4 by the freeze, so the showcase shows live labels in shadow mode. Step 5 after the showcase, since changing the head's input size during a freeze risks every node at once.

9. What is needed

1. A Roboflow account: download the carrier and package sets in COCO format into `D:\argus-data\roboflow\<name>\`, or set `ROBOFLOW_API_KEY`.
2. A team decision on switching the federated head to 35 features.
3. Access to the Hailo Dataflow Compiler (x86 Linux or WSL2) and confirmation of which Hailo chip ships.
4. Real per-site delivery windows and household cars, to replace the simulation assumptions.

10. Sources

- Stanford Cars, Hugging Face mirror: https://huggingface.co/datasets/Donghyun99/Stanford-Cars
- VMMRdb: https://github.com/faezetta/VMMRdb ; Tafazzoli, Frigui, Nishiyama, "A Large and Diverse Dataset for Improved Vehicle Make and Model Recognition", CVPR Workshops 2017

- Roboflow Packages dataset: https://public.roboflow.com/object-detection/packages-dataset
- Roboflow Universe (unverified pages): USPS Merge, Delivery Trucks, mailers and boxes 2, Envelope, Parcels and Boxes (links in section 2)
- MobileNetV3-Large weights: torchvision `MobileNet_V3_Large_Weights.IMAGENET1K_V2`